◆ DAY 3

后端开发、API 系统
数据库与模型接入

把 Day 2 智能客服接成可登录、可持久化、可真实对话的全栈系统

主讲：Tauh

课程地图·基础、架构与请求

01 开场 05

02 AI 时代的能力构建与项目架构 11

03 前后端如何通过请求协作 17

课程地图·环境、接口与对话

04 环境配置 40

05 FastAPI 项目初始化与接口开发 53

06 对话 API、模型接入与持久化 70

课程地图·联调、排错与验收

07 把 Day 2 前端接上 Day 3 后端 80

08 AI/Vibe Coding 常见工程问题与排错 88

09 课后作业与评分 97

10 环境安装命令附录 104

01

开场

从 Day 2 的前端页面出发，明确今天要完成的后端闭环

1.1	Day 2 已经完成了什么	06
1.2	今天要跨越的四道鸿沟	07
1.3	今日完成后的完整请求链路	08
1.4	今日学习目标	09
1.5	今日交付物	10

1.1 Day 2 已经完成了什么

识别一个“看起来完成”的假客服

- 已有智能客服官网和聊天内页：输入框、消息气泡、侧边栏、登录按钮等。
- 已具备 Node、前端框架、浏览器 F12、AI 编程工具的基本使用能力。
- 今天保留现有 UI，在当前项目上接入后端。
- 点击“发送”只是把文字追加到数组，没有发出网络请求。
- 回复来自 `setTimeout()` 或写死的示例文本，没有调用模型。

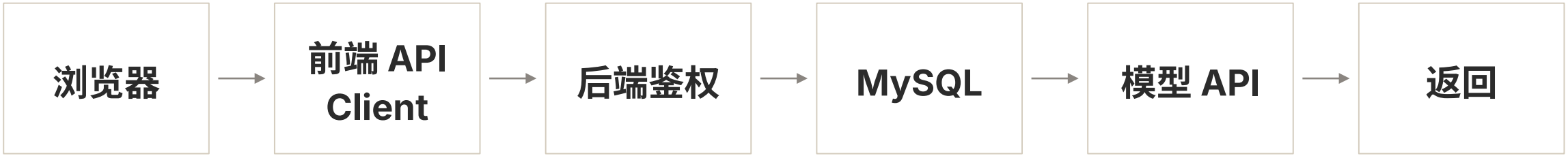
1.2 今天要跨越的四道鸿沟

- 页面状态→后端业务逻辑。
- 内存/浏览器存储→MySQL 持久化。
- 前端直连模型→后端安全调用模型。
- “我这里能跑”→有接口契约、错误处理和验收证据。

现场验证·学员要拿出证据

- 浏览器 Network 中出现真实请求
- 后端终端能看到同一请求的日志
- 刷新页面后数据仍可恢复

1.3 今日完成后的完整请求链路



链路 完整闭环：请求、持久化、模型调用、恢复历史

- 用户在浏览器输入问题。
- 前端向后端 API 发送 Token、会话 ID 和问题。
- 后端鉴权、校验会话归属、保存用户消息、调用大模型。
- 后端保存助手回复并返回/流式推送给前端。

1.4 今日学习目标

今日核心能力：看懂系统链路

- 能解释前端、后端、数据库和模型服务分别负责什么。
- 能安装并验证 Python、uv、MySQL 与 Navicat 开发环境。
- 能设计并调试 API，读懂请求、响应、状态码和错误日志。
- 能完成登录注册、会话隔离、数据库持久化和真实模型对话。
- 能指出入口文件、路由、业务服务、数据模型、配置和依赖的位置。
- 能沿着一次请求找到前端函数、后端路由、SQL 或模型调用。
- AI 负责写代码，但我们必须能理解“数据从哪里来、到哪里去、失败在哪里”。

1.5 今日交付物

全班沿 Python + FastAPI 主线完成同一个全栈闭环

- 一个前后端分离的智能客服项目。
- 一个 MySQL 数据库与可重复执行的迁移方案。
- 注册、登录、用户信息、会话列表、消息发送/读取接口。
- `.env.example`、README、接口测试证据和演示视频/截图。
- Python 3.12 与 uv：统一解释器、依赖和可复现环境。
- FastAPI：编写接口、生成 Swagger UI、连接 MySQL、接入模型。
- 主流技术栈保留选型认知；代码、排错与作业使用 Python + FastAPI。

02

AI 时代的能力构建与项目架构

AI 放大工程能力；项目架构决定产品可以走多远

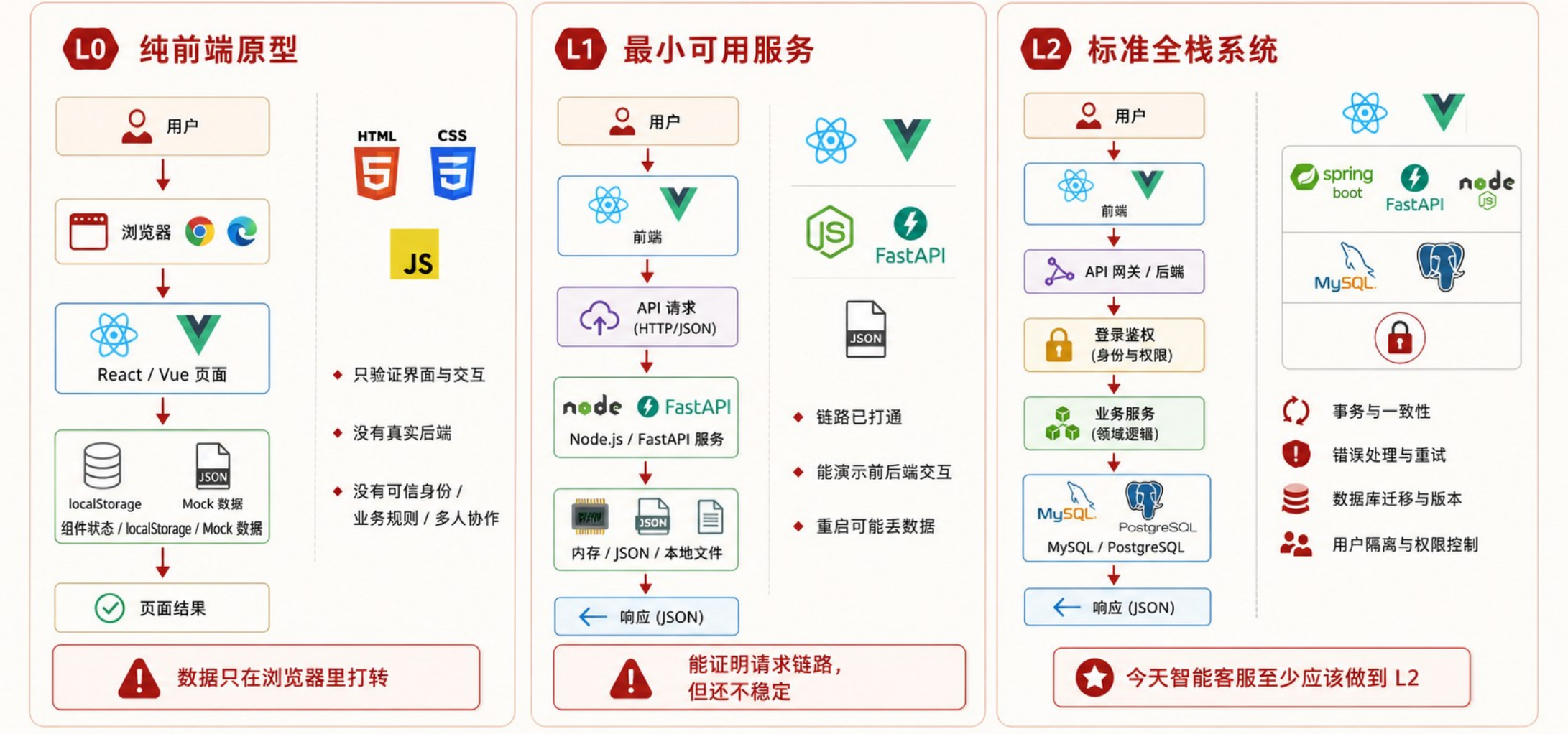
2.1	AI 会放大能力，也会放大知识盲区	12
2.2-1	L0–L2：从前端原型走到标准全栈闭环	13
2.2-2	L3：按真实瓶颈增强模块化单体	14
2.2-3	L4：业务边界扩大后拆分领域服务	15
2.2-4	L5：平台能力支撑多团队稳定交付	16

2.1 AI 会放大能力，也会放大知识盲区

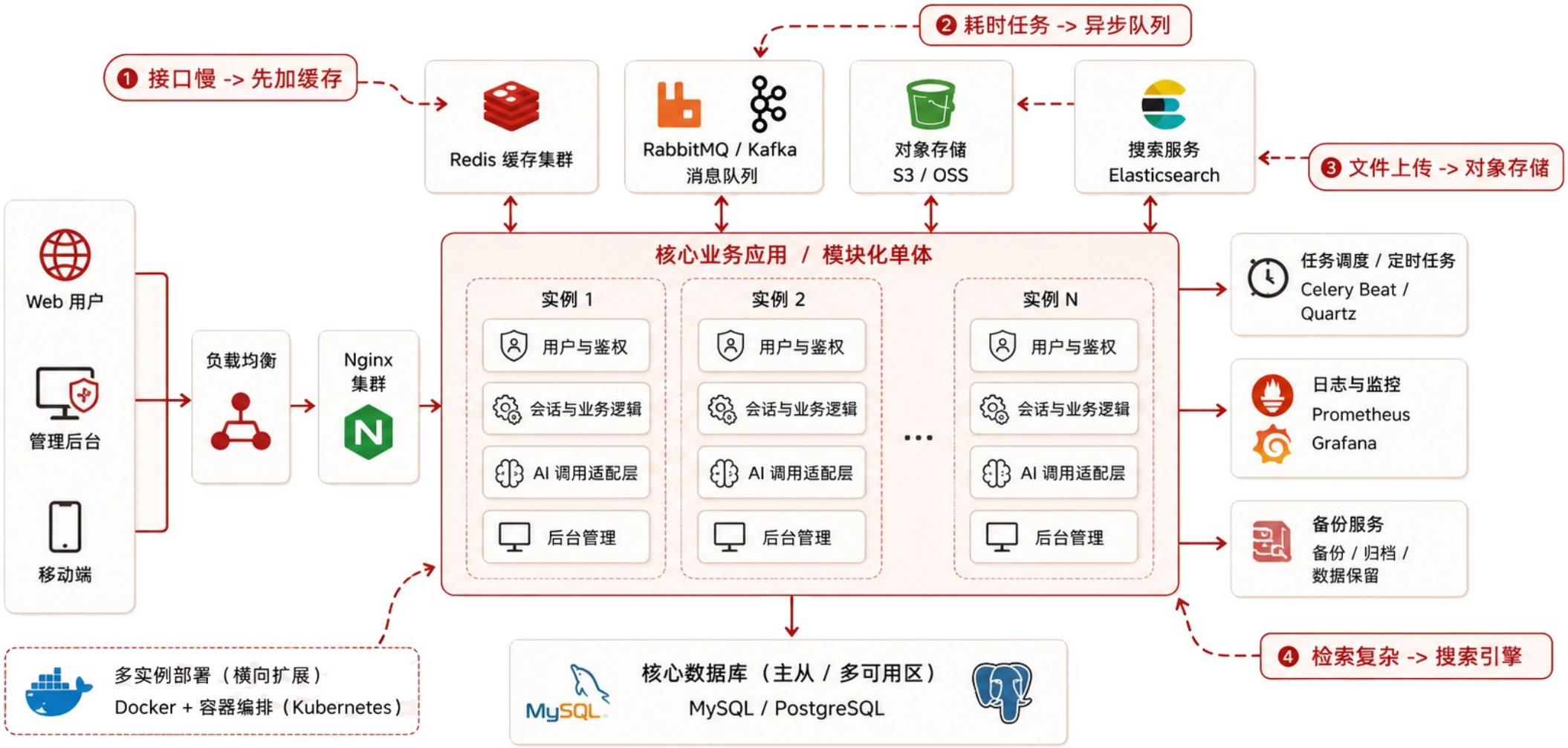
Vibe Coding 时代的能力分水岭



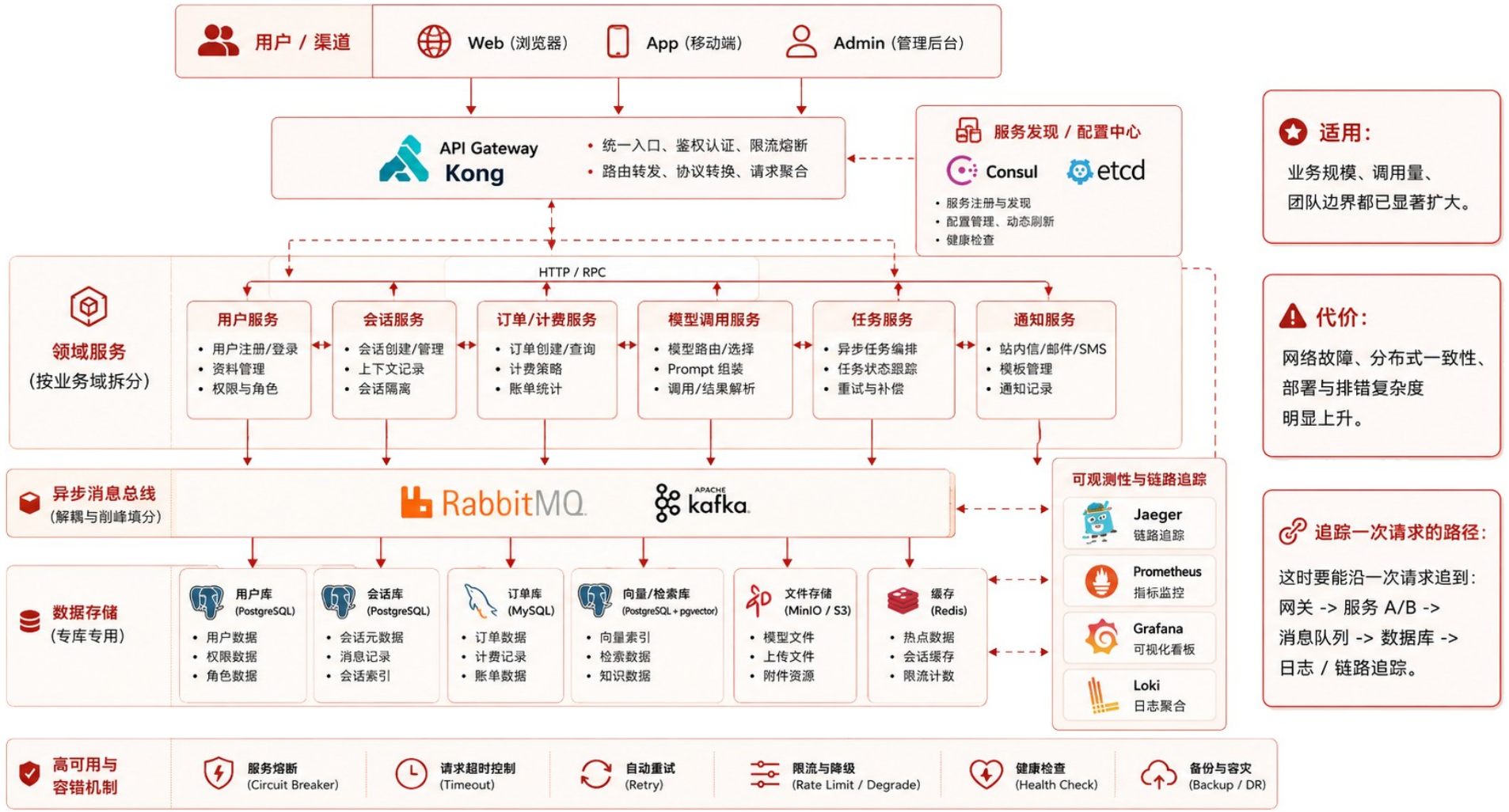
2.2-1 L0–L2：从前端原型走到标准全栈闭环



2.2-2 L3：按真实瓶颈增强模块化单体



2.2-3 L4：业务边界扩大后拆分领域服务



适用：

业务规模、调用量、
团队边界都已显著扩大。

代价：

网络故障、分布式一致性、
部署与排错复杂度
明显上升。

追踪一次请求的路径：

这时要能沿一次请求追到：
网关 -> 服务 A/B ->
消息队列 -> 数据库 ->
日志 / 链路追踪。

2.2-4 L5：平台能力支撑多团队稳定交付



L5 的核心不是“服务更多”，而是“让大量团队在统一治理下稳定、高效、可审计地交付服务”。

适用场景

- 大型组织，多团队协作，多业务域。
- 对稳定性、可靠性、合规性要求极高。
- 需要长期演进与标准化治理。

组织前提

- 具备平台工程团队与能力。
- DevOps / SRE 文化成熟。
- 统一规范、服务契约与治理体系落地。

代价与风险

- 平台建设与持续运营成本高。
- 治理复杂度高，需要持续投入。
- 过度平台化、流程过重可能拖慢交付。

03

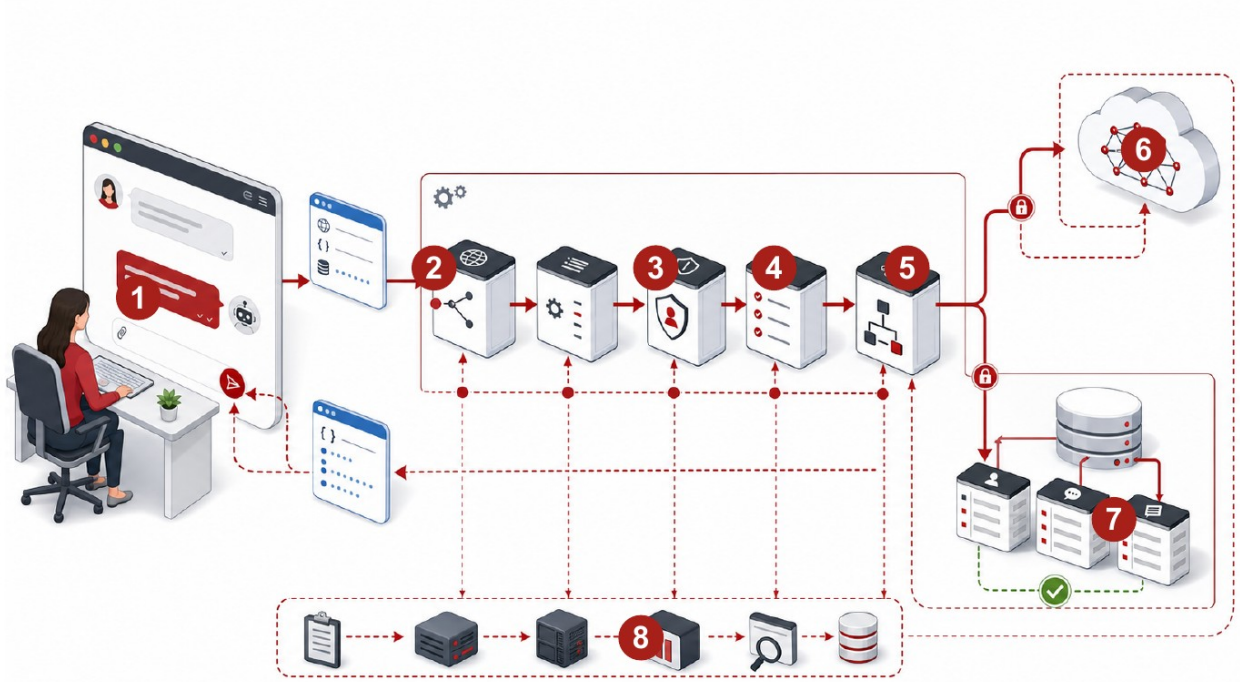
前后端如何通过请求协作

先理解谁和谁通信，再追踪一次请求怎样变成数据库中的真实记录

3.1	一次请求怎样穿过完整后端系统	18
3.2	前后端连接的本质是找到地址并交换消息	19
3.3	系统通信方式按交互模式分类	20
3.4	四种通信方式的节奏不同	21
3.5	本课程先掌握 HTTP 请求—响应闭环	22
3.6	一个 HTTP 请求由四部分组成	23
3.7	URL 把请求准确送到某个资源	24
3.8	HTTP 方法表达对资源的操作语义	25
3.9	Headers 保存请求上下文与控制信息	26
3.10	身份凭证进入请求后，后端仍要继续授权	27
3.11	Body 根据数据形态选择格式	28
3.12	把一条真实 HTTP 请求逐行读懂	29
3.13	HTTP 响应告诉前端结果与下一步动作	30
3.14	状态码把结果分成可行动的类别	31
3.15	后端按固定管线处理一次请求	32
3.16	后端把请求数据映射成可校验的输入	33
3.17	Service 编排一次完整的业务处理	34
3.18	一条客服消息穿过完整业务闭环	35
3.19	接口契约统一前端、后端与工具	36
3.20	沿证据链定位前后端问题	37
3.21	数据库把业务数据组织成可查询的结构	38
3.22	三张核心表保存身份、会话与消息	39

3.1 一次请求怎样穿过完整后端系统

前端只连接 API；后端统一控制身份、业务、数据库和模型



1 前端与 API Client
收集输入并构造 HTTP 请求。

2 路由与中间件
定位接口，建立 Request ID 与日志。

3 认证与授权
确认身份、权限和资源归属。

4 参数校验
检查类型、必填项和业务边界。

5 业务服务
编排事务、数据库和外部调用。

6 模型 API
密钥由后端保管，统一超时与重试。

7 数据库
保存用户、会话、消息与状态。

8 日志与追踪
沿同一请求定位故障位置。

3.2 前后端连接的本质是找到地址并交换消息

- 后端程序启动后监听一个端口，例如 FastAPI 的 8000。
- 客户端通过协议、主机、端口和路径找到后端接口。
- 前端发送请求；后端处理后返回响应，前端再更新页面。
- 本地开发常见组合：前端 `localhost:5173`，后端 `localhost:8000`。

概念	含义
IP / 域名	找到哪一台计算机
端口	找到计算机里的哪个服务进程
路径	找到服务中的哪个业务入口
Origin	协议、主机、端口的组合
CORS	浏览器对跨 Origin 请求的安全检查

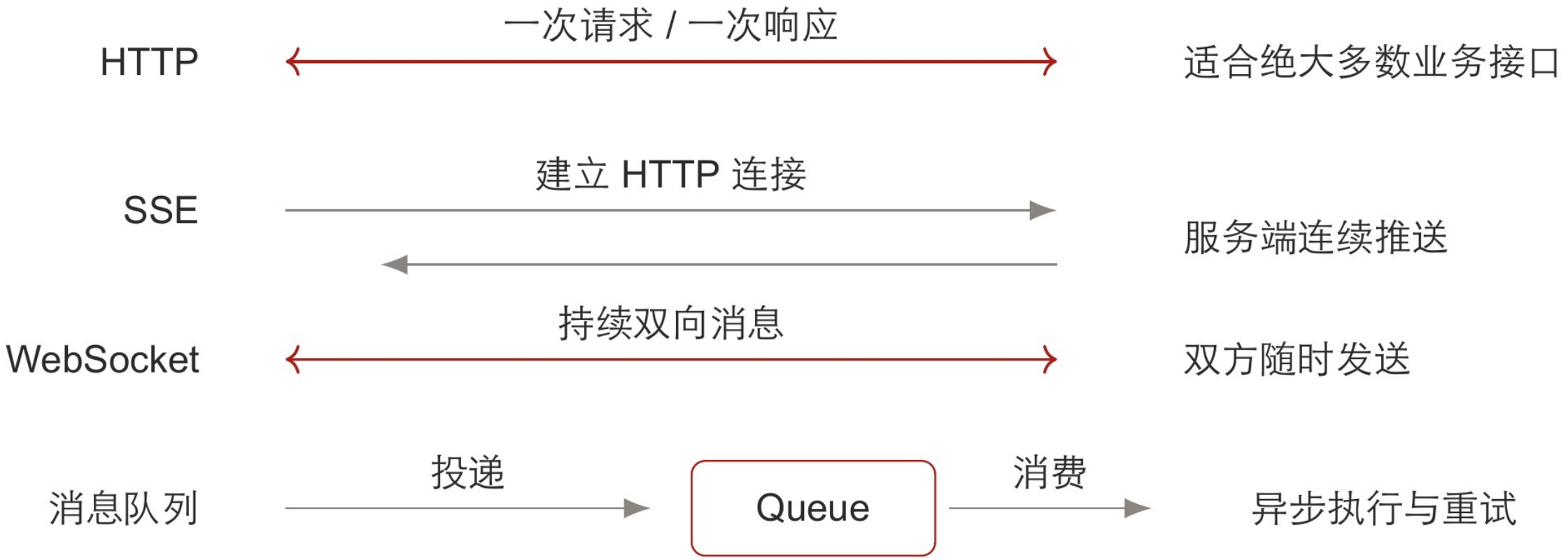
定位示例 `localhost:8000` = 本机后端服务地址

3.3 系统通信方式按交互模式分类

方式	交互模式	典型场景	初学者判断
HTTP / HTTPS	一次请求、一次响应	登录、CRUD、文件上传、模型调用	业务 API 的默认选择
SSE	服务器持续推送单向事件	大模型逐字输出、任务进度	浏览器接收流式文本很方便
WebSocket	长连接、双向实时通信	在线协作、游戏、即时消息	双方都要随时主动发消息
RPC / gRPC	像调用函数一样调用远端服务	内部服务间高性能通信	服务端工程常见，浏览器主线较少
消息队列	异步投递、稍后处理	邮件、报表、耗时任务、削峰	发送方无需等待立即完成

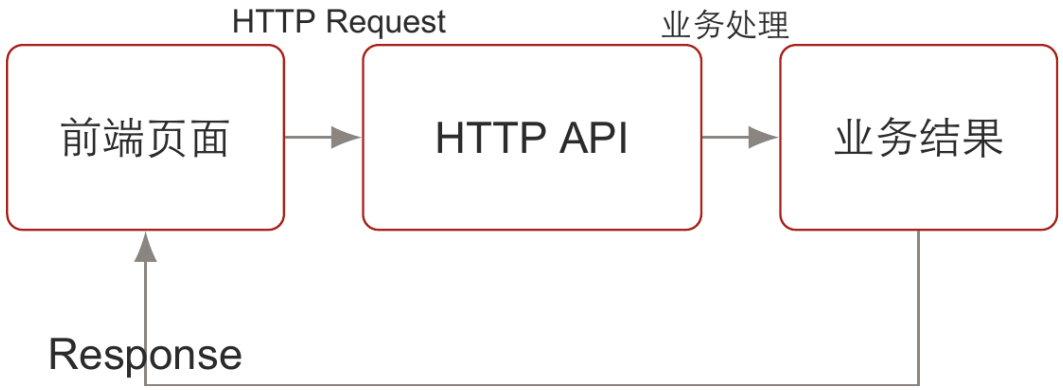
分类依据 是否等待响应、是否保持连接、谁能主动发送、是否允许稍后处理

3.4 四种通信方式的节奏不同



选型 CRUD 用 HTTP；模型流式输出用 SSE；强实时双向交互用 WebSocket

3.5 本课程先掌握 HTTP 请求—响应闭环

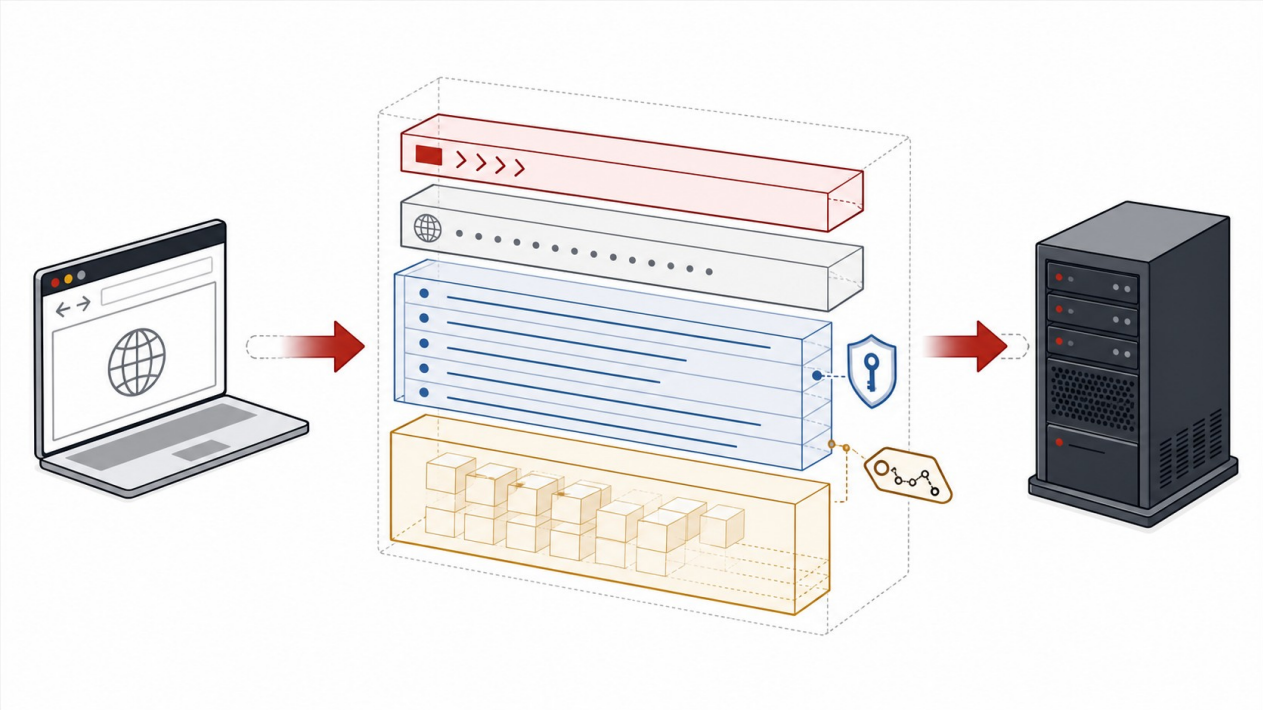


- 浏览器、移动端、脚本和服务器都能调用。
- 每次调用都有明确输入、输出和状态码。
- curl、Swagger、Apifox、Postman 都能独立验证。
- HTTPS 在 HTTP 外增加加密与服务端身份验证。

学习目标 看懂请求的来源、内容、目的与返回

3.6 一个 HTTP 请求由四部分组成

方法、URL、Headers 与 Body 共同表达调用意图



1. **方法 Method**：想查询、创建、修改还是删除。

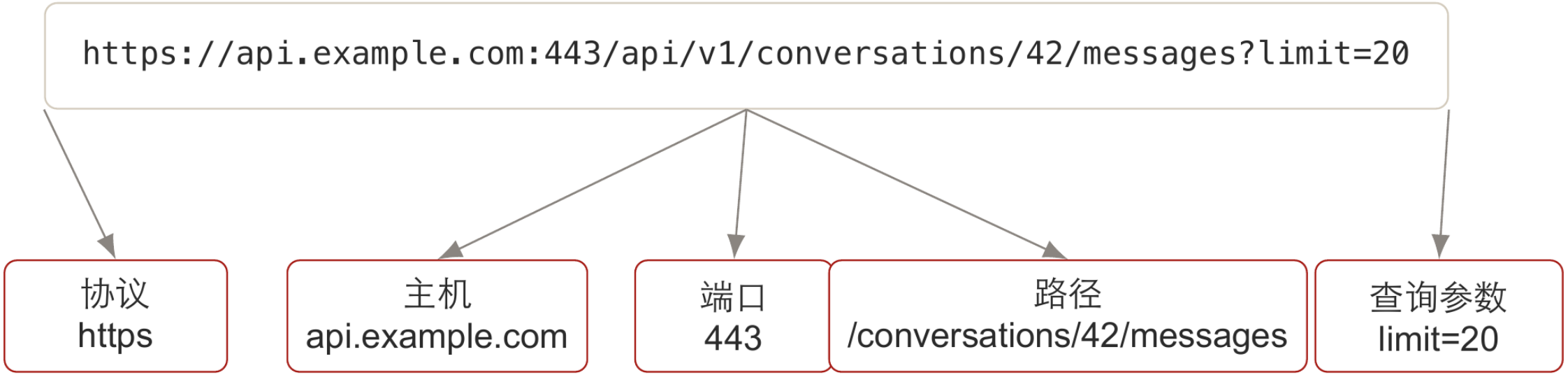
2. **URL**：目标服务、资源路径与查询条件。

3. **Headers**：内容类型、身份、来源、缓存与追踪元信息。

4. **Body**：真正提交的业务数据或文件内容。

记忆 方法说明动作，URL 指向资源，Headers 描述上下文，Body 承载业务内容

3.7 URL 把请求准确送到某个资源



数据位置 资源 ID 放 Path；筛选与分页放 Query

3.8 HTTP 方法表达对资源的操作语义

方法	常见意图	智能客服示例	重复调用要点
GET	查询资源	获取会话与消息列表	应保持只读
POST	创建资源或执行业务动作	创建会话、发送消息	可能重复创建或扣费
PUT	整体替换	替换完整用户配置	相同请求结果应一致
PATCH	局部修改	修改会话标题或状态	只更新明确字段
DELETE	删除或标记删除	删除会话	重复删除需定义结果

工程意识 对可能重试的创建与付费动作设计 Idempotency-Key 或业务去重键

3.9 Headers 保存请求上下文与控制信息

Header	存放什么	作用与注意事项
Content-Type	Body 的格式	`application/json`、`multipart/form-data`
Accept	客户端希望接收的格式	常见为 JSON 或事件流
Authorization	Bearer Token 等凭证	后端据此认证身份；日志需脱敏
Cookie	Session ID 等浏览器状态	浏览器自动携带；配置 SameSite 与 Secure
Origin	页面来源	后端 CORS 策略判断是否允许
X-Request-ID	一次请求的追踪 ID	串起网关、后端、数据库与日志
Idempotency-Key	业务去重标识	防止网络重试造成重复创建或重复扣费

原则 Headers 适合元信息；密码、Token 与 API Key 都不能写进 URL

3.10 身份凭证进入请求后，后端仍要继续授权

方式	请求放在哪里	后端怎样确认	常见场景
Bearer JWT	Authorization Header	验签并检查过期时间、签发方与受众	前后端分离 API、移动端
Cookie Session	Cookie Header	用 Session ID 查询服务端会话	同站 Web 应用、传统后台
API Key	自定义 Header	查询调用方、额度与权限范围	服务间调用、模型供应商 API

- 认证回答“你是谁”；授权继续判断“你能否访问这条会话或消息”。
- 后端从可信凭证得到当前用户 ID，再把它加入数据库查询条件。

安全 后端按当前身份校验资源归属

3.11 Body 根据数据形态选择格式

格式	Content-Type	适用数据
JSON	application/json	对象、数组、字符串、数字、布尔值；主流业务 API
表单	application/x-www-form-urlencoded	简单键值对、兼容传统表单
文件表单	multipart/form-data	图片、PDF 与同时提交的文字字段
二进制	application/octet-stream	单个原始文件流或协议数据
无 Body	无需 Content-Type	GET 查询、204 响应等没有正文的场景

校验 后端同时检查格式、字段类型、必填项、长度、文件大小与 MIME 类型

3.12 把一条真实 HTTP 请求逐行读懂

发送客服消息

```
POST /api/v1/conversations/42/messages HTTP/1.1
Host: localhost:8000
Content-Type: application/json
Authorization: Bearer eyJhbGciOi...
X-Request-ID: req_20260713_001
Idempotency-Key: msg_client_8848
{"content":" 如何申请退款? "}
```

1. POST 表示创建消息。
2. Path 中的 42 是会话 ID。
3. Token 证明调用者身份。
4. Request ID 用于跨系统查日志。
5. Idempotency-Key 防止重复发送。
6. JSON Body 保存用户问题。

3.13 HTTP 响应告诉前端结果与下一步动作

后端返回

```
HTTP/1.1 201 Created
Content-Type: application/json
X-Request-ID: req_20260713_001
{
  "id": 991,
  "role": "user",
  "content": " 如何申请退款? ",
  "created_at": "2026-07-13T15:30:00Z"
}
```

- 状态行：协议版本、状态码与简短描述。
- Headers：响应格式、缓存、跨域、追踪信息。
- Body：成功数据或结构化错误。
- 前端先判断状态码，再解析对应正文。

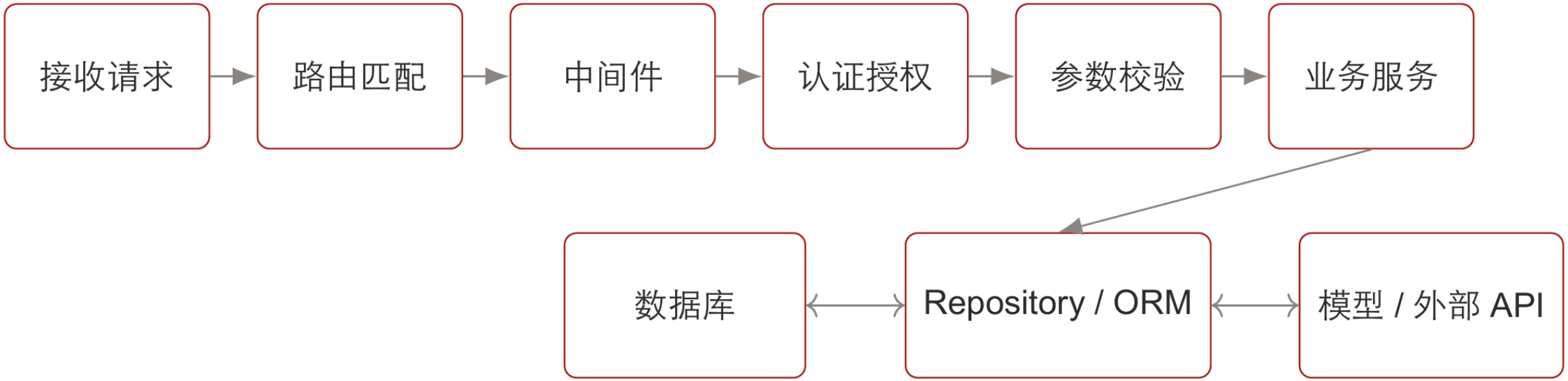
联调 请求与响应共享同一个 X-Request-ID，后端日志也记录该 ID

3.14 状态码把结果分成可行动的类别

类别	代表状态码	前端与调用方动作
成功	200 / 201 / 204	读取数据、更新页面；204 不读取正文
请求问题	400 / 409 / 422	修正字段、处理重复冲突、展示校验信息
身份权限	401 / 403 / 404	重新登录、提示无权限，或隐藏资源存在性
频率限制	429	读取 Retry-After，降低频率并执行有限重试
服务异常	500 / 502 / 503	保存 Request ID，查看后端日志，按策略重试或降级

错误正文 统一返回 code、message、details 与 request_id，方便前端展示和工程排查

3.15 后端按固定管线处理一次请求



返回 结果逐层转换为 JSON、状态码与响应 Headers

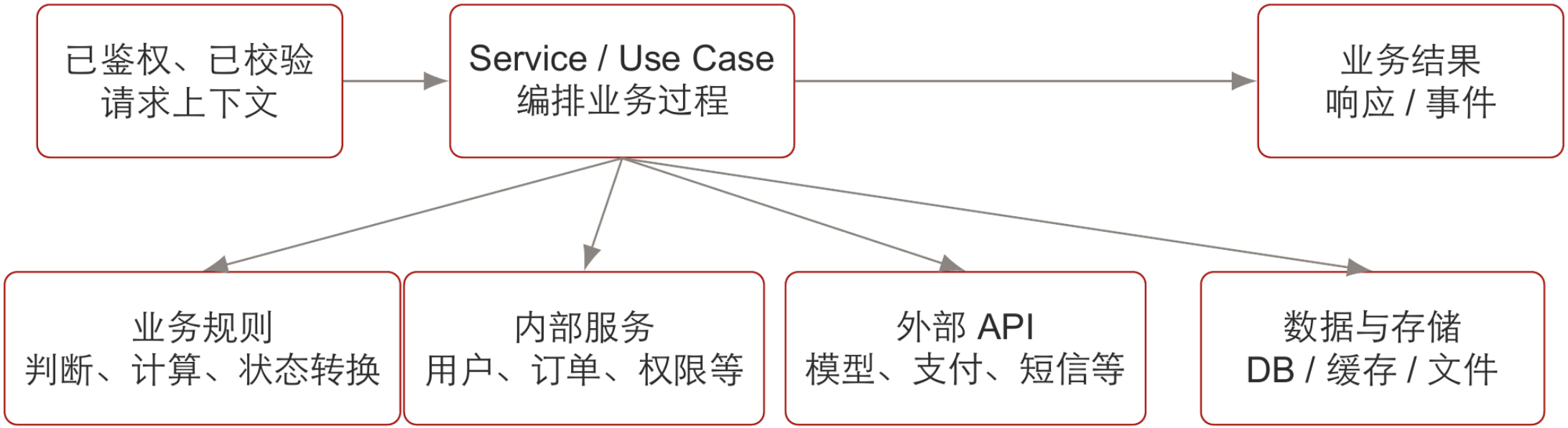
3.16 后端把请求数据映射成可校验的输入

来源	示例	后端处理
Path	`/conversations/42`	转成 conversation_id，并检查资源是否存在与归属
Query	`limit=20, cursor=...`	校验范围、默认值、排序与分页规则
Headers	Authorization / Request ID	建立当前身份、追踪上下文与幂等上下文
Body	JSON content / title	用 Schema/DTO 校验类型、长度、枚举和必填项
File	multipart 图片或 PDF	校验 MIME、大小、扩展名、恶意内容和权限

失败位置 字段问题在业务服务前返回 400/422

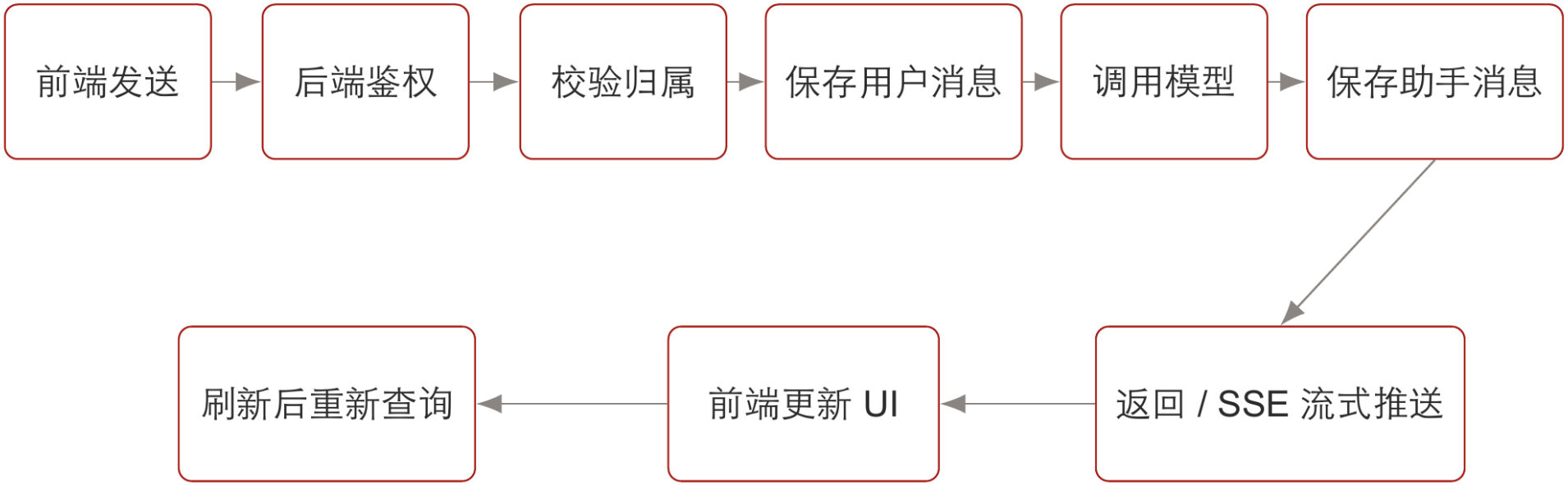
3.17 Service 编排一次完整的业务处理

规则、计算、内部服务、外部 API 与持久化按业务需要协作



- Service 对应一个业务用例，例如注册用户、发送消息、创建订单。
- 根据业务需要执行规则判断、数据计算、状态转换和内部模块协作。
- 调用外部 API 时处理鉴权、超时、重试、限流和幂等。
- 通过 ORM、Repository 或参数化 SQL 读写数据库，也可使用缓存和对象存储。

3.18 一条客服消息穿过完整业务闭环



- 用户消息先落库，模型失败时仍能看到真实请求历史。
- 助手回复完成后再落库；刷新页面从数据库恢复完整会话。

验收 刷新页面、重启后端后仍能恢复消息，才证明后端与数据库真的接通

3.19 接口契约统一前端、后端与工具

- OpenAPI 描述路径、方法、参数、Body Schema、响应与鉴权要求。
- 前端按契约构造请求，后端按契约验证并返回。
- JSON 字段名、类型、必填项、日期与错误结构保持稳定。
- 先用工具单独验证后端，再接页面，故障边界更清楚。

工具	主要用途
Swagger UI	查看 Schema、快速试调接口
Apifox	设计、Mock、调试、环境与自动化
Postman	Collection、环境、脚本与协作
curl	最小复现、README、服务器与 CI

安全 共享示例；真实密钥留在受控环境

3.20 沿证据链定位前后端问题

浏览器 Network
请求是否发出

→

curl / Swagger
能否独立复现

→

后端日志
路由、参数、异常

→

数据库查询
记录是否存在

- 保存 URL、方法、状态码、请求 Body 和响应 Body。
- 使用同一个 Request ID 检索后端日志。
- 用 SQL 客户端确认最终数据状态。
- 下一章安装 Python、uv、MySQL 与数据库客户端，把这条链路真正跑起来。

结论 请求连接前后端；后端把输入变成可信业务与持久化数据

3.21 数据库把业务数据组织成可查询的结构

先理解 Server、数据库、表、行与列的层级

MySQL Server
监听 3306

→

Database
customer_service

→

Table
users / messages

→

Row + Column
一条记录与字段

- 表结构规定字段名、类型、必填项和默认值；每行是一条业务记录。
- 主键唯一定位一行；外键表达表之间的归属关系。
- 索引加快常用查询；事务保证一组写入同时成功或同时回滚。
- SQL 完成增删改查；后端通常通过 ORM 生成并执行 SQL。

3.22 三张核心表保存身份、会话与消息

一对多关系让每条数据都能找到所属用户与上下文



- 注册：向 users 新增记录；登录：按邮箱查询并校验密码哈希。
- 创建会话：向 conversations 新增记录，并写入当前用户 ID。
- 发送消息：先写入 user 消息，模型返回后再写入 assistant 消息。
- 查询历史：按 conversation_id 排序读取；重启后端后记录仍在。

CRUD Create 新增·Read 查询·Update 修改·Delete 删除

04

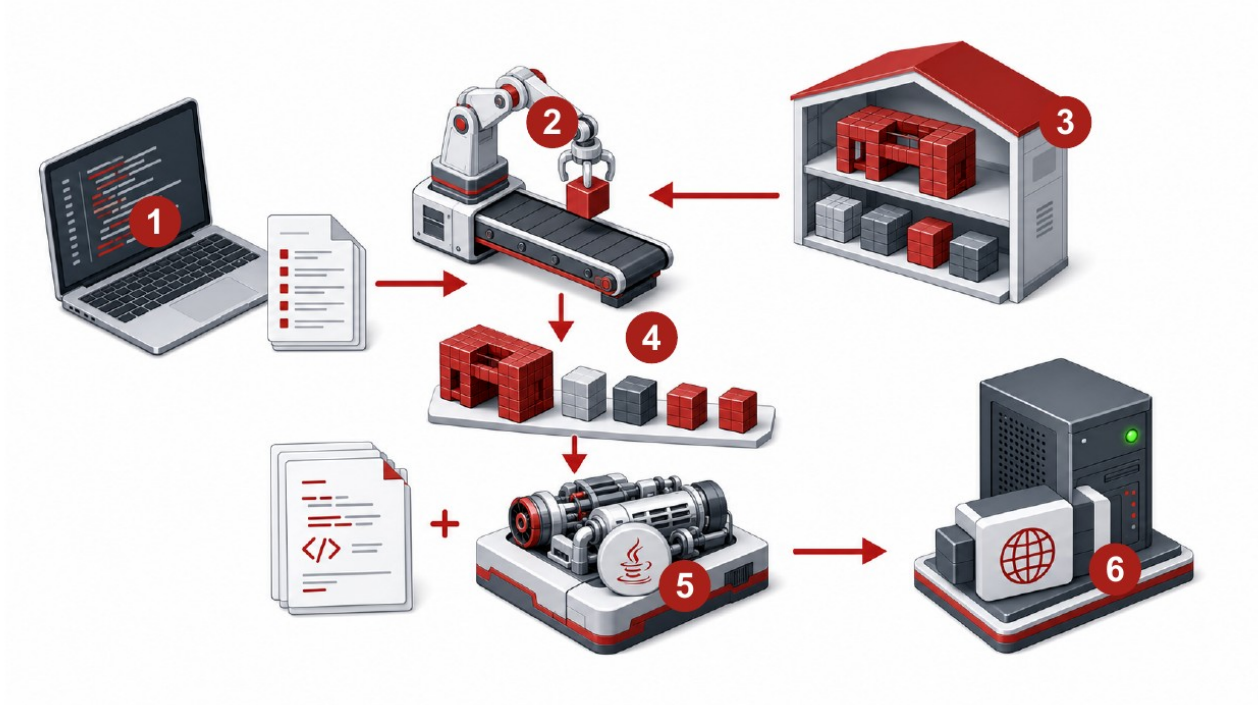
环境配置

统一完成 Python、uv、MySQL、Navicat 的本地配置与验证

4.1	包管理器、框架与运行时怎样协作	41
4.2	主流后端技术栈各有优势	42
4.3	环境配置先验证命令、版本、路径与服务	43
4.4	Python 为什么适合今日主线	44
4.5	用 uv 安装并固定 Python	45
4.6	虚拟环境把 Python 项目的依赖隔离开	46
4.7	Windows：安装 MySQL Server	47
4.8	macOS：安装 MySQL Server	48
4.9	应用账号与 root 必须分开	49
4.10	环境配置问题按证据顺序排查	50
4.11	环境阶段小验收	51
4.12	Python 项目的依赖清单与锁文件	52

4.1 包管理器、框架与运行时怎样协作

包管理器准备依赖，运行时执行代码与依赖



1 项目源码与依赖清单

开发者声明项目需要哪些包和版本。

2 包管理器

读取清单、解析版本、下载并锁定依赖。

3 包仓库

集中保存可下载的框架与第三方库。

4 框架与依赖包

框架提供项目骨架；库提供具体能力。

5 运行时 / JDK

加载、解释或编译代码与依赖。

6 运行中的后端

启动端口，开始接收 HTTP 请求。

4.2 主流后端技术栈各有优势

本课使用 Python + FastAPI，先掌握共同的后端概念

维度	Python + FastAPI	TypeScript + NestJS	Java + Spring Boot
运行与依赖	Python ·uv	Node.js ·pnpm	JDK ·Maven / Gradle
优势	AI 生态丰富，开发验证快	前后端语言统一，模块化清晰	企业生态成熟，长期治理能力强
典型场景	AI 应用、数据 API、快速产品验证	SaaS、BFF、实时 Web 服务	ERP、金融、政企核心业务
常见代价	大项目要主动约束类型与目录	要理解异步 I/O 与 npm 生态	概念较多，启动和构建更重
本课安排	完整安装、编码、调试与作业	了解选型场景	了解选型场景

主线 Python 3.12 + uv + FastAPI + MySQL

4.3 环境配置先验证命令、版本、路径与服务

- `git --version`：确认代码版本管理工具可用。
- `uv --version`：确认 Python 项目与包管理器可用。
- `uv run python --version`：确认项目实际使用 Python 3.12。
- `mysql --version`：确认客户端存在；连接查询验证 Server 正常。
- Navicat 能建立本地连接、看到数据库并执行 SQL。

环境配置自检

```
git --version
uv --version
uv run python --version
mysql --version
mysql -u root -p
SELECT VERSION();
```

证据 版本正确、实际路径明确、MySQL 服务可连接、Navicat 可查询

4.4 Python 为什么适合今日主线

安装 uv：Python 的项目与包管理器

- 语法简洁、FastAPI 自动生成接口文档，与 AI/RAG 生态衔接自然。
- Python 解释器与第三方包是两回事；有 Python 不代表装好了 FastAPI。
- 项目使用 Python 3.12，兼顾兼容性与可复现性。
- macOS/Linux：使用 uv 官方安装脚本或 `brew install uv`。
- Windows PowerShell：使用 uv 官方 PowerShell 安装命令或 `winget install --id=astral-sh.uv -e`。
- 重开终端后执行 `uv --version`；找不到命令时先查 PATH，再决定是否重装。

落点 重开终端后执行 `uv --version`；找不到命令时先查 PATH，再决定是否重装。

4.5 用 uv 安装并固定 Python

创建 Python 后端项目

- `uv python install 3.12`：安装指定的 Python 3.12。
- `uv python pin 3.12`：在项目中写入版本约束。
- `uv run python --version`：确认当前项目实际使用的解释器。
- 解释系统 Python、项目 Python、IDE 解释器选错的常见问题。
- `uv init backend-python --python 3.12`，进入目录后管理项目。
- `uv add "fastapi[standard]" sqlmodel alembic pymysql pydantic-settings`。
- 再添加认证与模型依赖：`pwdlib[argon2]`、`pyjwt`、`python-multipart`、模型 SDK。
- `pyproject.toml` 描述依赖，`uv.lock` 锁定可复现版本，都应提交 Git。

落点 `pyproject.toml` 描述依赖，`uv.lock` 锁定可复现版本，都应提交 Git。

4.6 虚拟环境把 Python 项目的依赖隔离开

- 每个项目拥有独立依赖，避免全局安装互相污染。
- uv 可自动管理 `.venv`；`uv run ...` 不依赖手动激活环境。
- `.venv` 体积大且可重建，应放进 `.gitignore`。

现场验证·学员要拿出证据

- 检查运行时与包管理器版本
- 确认实际执行文件路径
- 启动/health 并验证端口

落点 `.venv` 体积大且可重建，应放进 `.gitignore`。

4.7 Windows：安装 MySQL Server

完成服务配置并记录连接参数

- 课程使用 MySQL 8.4 LTS，生产数据库部署在 Day 5 展开。
- 从 MySQL 官方渠道安装 8.4 LTS 64 位版本，使用 Configurator 配置服务。
- 记录 root 密码、端口 3306、Windows 服务名和字符集。

4.8 macOS：安装 MySQL Server

用 Navicat 连接、浏览表并执行 SQL

- 可使用官方 DMG/PKG 或 Homebrew；选定一种安装路线后使用对应的服务管理命令。
- 学会启动、停止、查看服务状态；记录 root 密码和端口。
- `mysql --version` 只说明客户端存在，还要执行连接查询验证 Server。
- 命令行客户端必须会基本连接和执行 SQL，便于服务器环境排错。
- Navicat 用于保存连接、浏览表结构、查看数据和执行查询；命令行仍用于排错。

4.9 应用账号与 root 必须分开

创建项目数据库与应用账号

- root 用于数据库管理；项目创建独立账号，例如 `customer_service_app`。
- 只授予该项目数据库所需权限，限制来源为本机开发环境。
- 数据库账号密码放环境变量，不写在源码或提交记录中。
- 创建 `customer_service` 数据库，字符集使用 `utf8mb4`。
- 创建本地应用用户，授权该数据库的必要权限。
- 用应用账号重新连接并执行读写测试，证明权限配置有效。

落点 用应用账号重新连接并执行读写测试，证明权限配置有效。

4.10 环境配置问题按证据顺序排查

同一类故障沿固定顺序收集证据，避免让 AI 随机试命令

现象	先检查	常用证据
命令不存在	安装是否完成、终端是否重开、PATH	which / where / Get-Command
版本不对	Python 是否为 3.12、IDE 是否同源	uv run python –version、实际路径
包不存在	当前目录、锁文件、包管理器、虚拟环境	uv tree / pyproject.toml / uv.lock
端口占用	占用进程与服务端口配置	lsof / netstat / 后端启动日志
网络失败	URL、代理、证书、镜像与防火墙	curl -v / 状态码 / DNS

底线 保留第一段错误、完整命令和环境信息，再向 AI 提问

4.11 环境阶段小验收

- FastAPI 项目有一个可访问的 `/health`。
- 学生能说出启动命令、停止方式、端口和入口文件。
- MySQL 服务可连接，Navicat 能查看 `customer_service`，FastAPI 能读取 `DATABASE_URL`。

现场验证·学员要拿出证据

- 检查运行时与包管理器版本
- 确认实际执行文件路径
- 启动/health 并验证端口

排错 每个检查都保留命令、版本、连接结果或截图。

4.12 Python 项目的依赖清单与锁文件

pyproject.toml 说明需要什么，uv.lock 固定实际安装版本

- pyproject.toml 记录项目名称、Python 版本范围和直接依赖。
- uv.lock 锁定直接与间接依赖的精确版本，保证不同电脑安装结果一致。
- 使用 uv add 增加依赖、uv remove 删除依赖、uv sync 按锁文件同步环境。
- uv tree 查看依赖树；报错时先确认当前目录、Python 版本和实际虚拟环境。

Python 依赖管理

```
uv add 'fastapi[standard]'
uv add sqlmodel alembic pymysql
uv add pydantic-settings
uv add 'pwdlib[argon2]' pyjwt
uv sync
uv tree
```

提交 Git pyproject.toml 与 uv.lock 提交；.venv 与.env 忽略

05

FastAPI 项目初始化与接口开发

初始化项目，用 Swagger 验证接口，再完成登录注册与会话 CRUD

5.1	初始化 FastAPI 项目并跑通第一个请求	54
5.2	推荐项目目录	55
5.3	配置与环境变量	56
5.4	数据库 Session 生命周期	57
5.5	依赖注入的初步理解	58
5.6	OpenAPI/Swagger 是活的接口说明书	59
5.7	健康检查与就绪检查	60
5.8	认证确认身份，授权确认能做什么	61
5.9	注册接口设计	62
5.10	登录接口设计	63
5.11	JWT：可读载荷与数字签名	64
5.12	前端如何携带身份	65
5.13	对象级权限是 AI 最容易漏掉的漏洞	66
5.14	401、403、404 如何选择	67
5.15	CORS 的错误做法	68
5.16	会话 API 清单	69

5.1 初始化 FastAPI 项目并跑通第一个请求

先得到可启动、可访问、可调试的最小工程

1. 创建项目并固定 Python 3.12。
2. 安装 FastAPI、数据库、迁移、配置与认证依赖。
3. 创建 `app/main.py`，实现 `GET /health`。
4. 启动开发服务器，观察端口与自动重载日志。
5. 用浏览器、Swagger UI 和 curl 三种方式验证。

初始化与启动

```
uv init customer-service-backend --python 3.12
cd customer-service-backend
uv add 'fastapi[standard]' sqlmodel alembic pymysql
uv add pydantic-settings 'pwdlib[argon2]' pyjwt
uv run fastapi dev app/main.py
curl http://localhost:8000/health
```

入口 http://localhost:8000/docs · /openapi.json · /health

5.2 推荐项目目录

- app/main.py：应用入口；api/：路由；schemas/：输入输出模型。
- models/：ORM；services/：业务；repositories/：数据访问。
- core/config.py：配置；db/：连接与会话；clients/：模型供应商。
- tests/、alembic/、.env.example、README。

推荐目录

```
app/
  main.py        # 入口
  api/           # Router
  services/      # 业务流程
  models/        # ORM
  schemas/       # 输入输出
  core/          # 配置与安全
  db/            # Session / migration
tests/
.env.example
README.md
```

验收 命令、版本、端口和实际执行文件位置都能被现场验证

5.3 配置与环境变量

数据库连接 URL 的组成

- DATABASE_URL、MODEL_API_KEY、MODEL_BASE_URL、MODEL_NAME、JWT_SECRET。
- 本地 .env 不提交；.env.example 只写字段名和安全示例。
- 应用启动时校验必需配置，缺失就立即给出清晰错误。
- 驱动、用户名、密码、主机、端口、数据库名和字符集。
- 密码含 @、:、/ 等字符时需要 URL 编码，常见于“明明密码正确却连接失败”。
- 本地数据库主机通常为 localhost；Docker 环境使用 Compose 服务名，Day 5 实操。

落点 本地数据库主机通常为 localhost；Docker 环境使用 Compose 服务名，Day 5 实操。

5.4 数据库 Session 生命周期

输入 Schema、数据库 Model、输出 Schema 不应完全混用

- 每个请求获取独立 Session，用完提交/回滚并关闭。
- 不把一个全局 Session 永久复用给所有请求。
- 异常发生时确保回滚，避免连接处于错误事务状态。
- 注册输入包含明文密码，但数据库只保存 `password_hash`。
- 用户输出不能把 `password_hash` 返回给前端。
- 服务端控制可写字段，避免客户端上传 `is_admin: true` 之类的越权字段。

落点 服务端控制可写字段，避免客户端上传 `is_admin: true` 之类的越权字段。

5.5 依赖注入的初步理解

统一响应与错误结构

- 路由声明需要数据库 Session、当前用户或配置，框架负责提供。
- 认证逻辑集中复用，不在每个接口复制 Token 解析代码。
- 便于测试时替换数据库或模型 Client。
- 成功响应保持稳定字段；错误至少包含机器可读 code 与人类可读 message。
- 不把完整异常堆栈、SQL、密钥或供应商响应直接发给浏览器。
- 前端根据稳定的 HTTP 状态码和机器可读错误码处理异常。

落点 前端根据稳定的 HTTP 状态码和机器可读错误码处理异常。

5.6 OpenAPI/Swagger 是活的接口说明书

- FastAPI 根据路由和 Schema 自动生成 /docs。
- 在 Swagger 中先完成注册、获取 Token、授权、创建会话、发送消息。
- 文档能打开不代表业务正确；仍需校验状态码、数据库和权限。

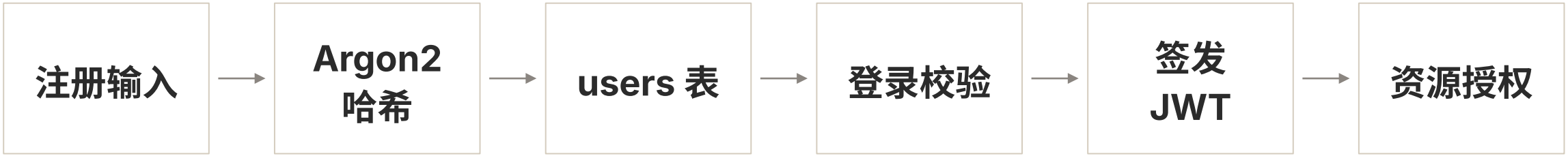
5.7 健康检查与就绪检查

结构化日志基础

- /health 表明进程活着；可增加数据库探测确认依赖可用。
- 不在健康检查中调用昂贵的大模型 API。
- Day 5 部署和监控会使用这些端点，因此今天先规范命名与响应。
- 记录时间、级别、请求 ID、路径、耗时、状态码和错误类别。
- 不打印密码、完整 Token、API Key、敏感聊天内容。
- 本地先学会从浏览器 Network → 后端日志 → SQL/模型响应顺序排错。

落点 本地先学会从浏览器 Network → 后端日志 → SQL/模型响应顺序排错。

5.8 认证确认身份，授权确认能做什么



链路 身份、Token 与资源归属是连续链路

- 认证 Authentication：你是谁，例如邮箱和密码登录。
- 授权 Authorization：你能做什么，例如只能读取自己的会话。
- Token 证明用户身份；每个 `conversation_id` 仍需执行资源归属校验。

5.9 注册接口设计

密码绝不能明文保存

- POST /api/v1/auth/register 接收邮箱、密码、昵称。
- 校验格式与强度；邮箱规范化；数据库唯一约束兜底。
- 成功返回 201 和安全用户信息；重复邮箱返回 409。
- 密码使用专门的慢哈希算法加盐保存；课程采用 Argon2。
- 登录时比较哈希，不需要也不能“解密回原密码”。
- 普通 SHA-256 速度太快，不适合直接作为密码存储方案。

落点 普通 SHA-256 速度太快，不适合直接作为密码存储方案。

5.10 登录接口设计

- POST /api/v1/auth/login 校验账号状态与密码哈希。
- 成功签发有过期时间的访问 Token；失败统一返回 401，避免泄露账号是否存在。
- 登录请求是明文密码传输，因此生产必须使用 HTTPS；本地仅在 localhost 演示。

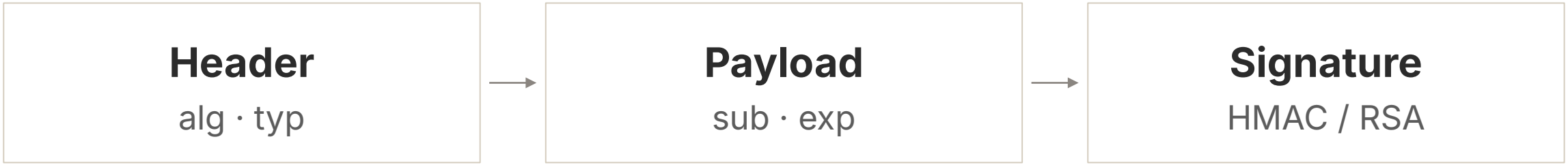
现场验证·学员要拿出证据

- 错误密码返回 401
- 过期或伪造 Token 被拒绝
- 跨用户资源访问返回 403/404

落点 登录请求是明文密码传输，因此生产必须使用 HTTPS；本地仅在 localhost 演示。

5.11 JWT：可读载荷与数字签名

JWT 的载荷可读，可信性来自签名与服务端校验



- JWT Payload 可以被读取，不能放密码、API Key 或敏感资料。
- 服务器用密钥验证 Token 是否被篡改，并检查 exp 等声明。
- sub 放稳定的用户标识；Token 需要到期时间。

安全 Token 不保存密码；后端校验签名、过期时间、签发方与资源归属

5.12 前端如何携带身份

当前用户接口

- Bearer 方案：`Authorization: Bearer <token>`。
- 课堂可先用易理解方案完成闭环；解释 `localStorage` Token 面临 XSS 风险。
- 生产常采用更完整的短期 Access Token、Refresh Token 或 HttpOnly/Secure Cookie 策略。
- `GET /api/v1/users/me` 从已验证 Token 中获取用户身份，忽略客户端自报的 `user_id`。
- 前端启动时通过该接口恢复登录状态与用户资料。
- 无效/过期 Token 返回 `401`，前端清理状态并跳转登录。

落点 无效/过期 Token 返回 `401`，前端清理状态并跳转登录。

5.13 对象级权限是 AI 最容易漏掉的漏洞

知道 conversation_id 不代表有权访问这条会话

危险写法

只按 id 查询：`SELECT * FROM conversations WHERE id=?`

- 登录成功后仍可枚举别人的资源 id。
- AI 常只加 JWT 中间件，没有校验对象归属。

正确边界

同时按 id 与当前用户查询：`WHERE id=? AND user_id=?`

- 更新、删除、上传、读取历史都执行同一规则。
- 对外可返回 404，减少资源存在性泄露。

验收 用 A 用户创建会话，再用 B 用户 Token 访问，必须失败且无数据泄露

5.14 401、403、404 如何选择

CORS 为什么出现

- 未认证用 401；已认证但无角色权限通常用 403。
- 对其他用户的私有资源可返回 404，避免暴露资源确实存在。
- 课程项目要保持同一种策略并写入接口契约。
- 浏览器同源策略按协议、主机、端口判断 Origin。
- Day 2 前端 `localhost:5173` 调 Day 3 后端 `localhost:8000` 属于跨源请求。
- 后端明确配置允许的本地前端 Origin、方法和 Headers；前端保持正常 CORS 请求模式。

落点 后端明确配置允许的本地前端 Origin、方法和 Headers；前端保持正常 CORS 请求模式。

5.15 CORS 的错误做法

认证模块验收

- 带 Cookie/凭据时不能简单使用任意来源 *。
- 不把所有域名、方法、Headers 永久开放来“消除报错”。
- CORS 约束浏览器跨源读取；API 身份认证和授权需要独立实现。
- 注册成功、重复邮箱冲突、弱/非法输入被拒绝。
- 正确密码登录成功，错误密码返回 401，Token 过期被拒绝。
- A 用户不能访问 B 用户的会话，即使猜到了会话 ID。

落点 A 用户不能访问 B 用户的会话，即使猜到了会话 ID。

5.16 会话 API 清单

- POST /api/v1/conversations 创建会话。
- GET /api/v1/conversations 查询当前用户会话列表并分页。
- GET /api/v1/conversations/{id} 返回会话及消息。
- PATCH/DELETE 可作为提高项；所有接口必须校验归属。

现场验证·学员要拿出证据

- 用户消息先写入数据库
- 模型调用有超时与失败分支
- 助手消息落库后可刷新恢复

落点 PATCH/DELETE 可作为提高项；所有接口必须校验归属。

06

对话 API、模型接入与持久化

会话 CRUD 跑通后，接入模型并保存完整对话历史

6.1	发送消息 API 是整套系统的主链路	71
6.2	模型 API Key 只能由后端保管	72
6.3	Chat 消息中的 role	73
6.4	先保存用户消息，再调用模型	74
6.5	流式输出为什么体验更好	75
6.6	流式接口需要明确的事件协议	76
6.7	模型调用的超时、重试与退避	77
6.8	模型输出的校验边界	78
6.9	历史上下文过长怎么办	79

6.1 发送消息 API 是整套系统的主链路



链路 一次消息从校验走到持久化

- 校验登录、会话归属、内容长度和幂等上下文。
- 保存 user 消息，构造上下文，调用模型。
- 保存 assistant 消息，更新会话时间，返回或流式输出。
- 失败时记录可恢复状态，返回对应错误码和重试信息。

6.2 模型 API Key 只能由后端保管

兼容式模型 Client 与供应商隔离

- 浏览器中的源码、Network 和打包产物都能被用户查看。
- 前端直连会暴露密钥、无法统一限流、审计、重试和成本控制。
- 正确链路是浏览器调用自己的后端，后端再调用模型供应商。
- 把 `base_url`、`api_key`、`model` 作为环境配置。
- 通过独立 `ModelClient` 封装供应商 SDK，不让路由直接散落模型调用代码。
- DeepSeek/GLM 等具体模型能力会变化，课堂模板要能通过配置替换。

落点 DeepSeek/GLM 等具体模型能力会变化，课堂模板要能通过配置替换。

6.3 Chat 消息中的 role

模型常用参数的基础认知

- system：应用规则和角色边界；user：用户输入；assistant：历史回复。
- 多轮对话必须按顺序提供必要历史，否则模型“失忆”。
- 数据库保存完整历史；发送给模型的上下文根据窗口与成本选取必要内容。
- 模型名、最大输出、温度/随机性、停止条件等。
- 不迷信统一“最佳温度”；客服更强调稳定、可控和可追踪。
- 参数名和可用范围以供应商当前文档为准，不让 AI 凭印象编造。

落点 参数名和可用范围以供应商当前文档为准，不让 AI 凭印象编造。

6.4 先保存用户消息，再调用模型

事务边界与外部 API

- 模型调用前结束数据库事务，避免长时间占用连接。
- 推荐保存 user 消息并标记状态，再调用模型；成功后保存 assistant 消息。
- 失败时把该轮标记为 `failed`，允许用户重试且不丢失输入。
- 数据库事务适合保护一组快速、本地的数据变更。
- 外部模型调用慢且可能超时，不应把数据库锁长期带到网络调用结束。
- 生产可用任务队列/事件进一步解耦；本地先实现清晰状态机。

落点 生产可用任务队列/事件进一步解耦；本地先实现清晰状态机。

6.5 流式输出为什么体验更好

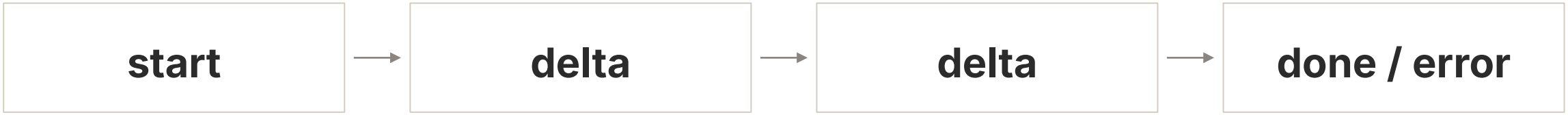
- 非流式：等待完整回复后一次返回；实现简单但首字等待长。
- 流式：模型产生 Token 时逐步推送，用户更早看到内容。
- 浏览器可使用 Fetch 流或 SSE；WebSocket 留给真正双向实时场景，不为炫技强上。

现场验证·学员要拿出证据

- 用户消息先写入数据库
- 模型调用有超时与失败分支
- 助手消息落库后可刷新恢复

落点 浏览器可使用 Fetch 流或 SSE；WebSocket 留给真正双向实时场景，不为炫技强上。

6.6 流式接口需要明确的事件协议



链路 每个事件有边界，网络分块不等于完整 JSON

- 约定事件类型：`start`、`delta`、`done`、`error`。
- 每个 chunk 必须有明确边界，不能假设一次网络读取等于一个完整 JSON。
- done 时提供 message ID、用量等最终元数据；中途断开要可识别。

6.7 模型调用的超时、重试与退避

限流、成本与资源保护

- 连接超时和总超时必须设置；不能让请求无限挂起。
- 只对适合的临时错误重试，并使用指数退避与次数上限。
- 用户重复点击与后端自动重试可能产生重复消息，需要请求 ID/幂等思维。
- 按用户/IP 做基础频率和并发限制，限制输入长度与历史轮数。
- 记录模型、Token/用量、耗时和结果状态，便于成本核算与排错。
- 429 给前端可理解提示，并采用有上限的退避重试。

落点 429 给前端可理解提示，并采用有上限的退避重试。

6.8 模型输出的校验边界

对话标题的生成策略

- 模型可能幻觉、遗漏或生成不合规内容。
- 业务关键数据必须来自数据库/API/RAG，不靠模型“记得”。
- 输出展示要防止把模型生成的 HTML/Markdown 当作无条件可信代码执行。
- 最简单：取首条用户消息的截断文本，零额外模型成本。
- 提高项：异步让模型总结短标题，失败不影响聊天主流程。
- 标题同样需要保存到数据库，不能每次刷新重新随机生成。

落点 标题同样需要保存到数据库，不能每次刷新重新随机生成。

6.9 历史上下文过长怎么办

对话闭环验收

- 先限制最近 N 轮/Token 预算，不能把数据库所有消息无限发送。
- 可用摘要压缩早期对话；原始消息仍保存在数据库。
- Day 4 的 RAG 处理外部知识检索；聊天历史压缩与管理需要单独设计。
- Network 中能看到真实后端请求；后端日志能看到模型调用。
- MySQL 同时出现 user 和 assistant 消息，状态正确、顺序正确。
- 刷新、退出再登录、重启后端后，历史仍可从数据库加载。

落点 刷新、退出再登录、重启后端后，历史仍可从数据库加载。

07

把 Day 2 前端接上 Day 3 后端

先验证接口，再接页面；联调问题逐层定位

7.1	前端改造集中在 API、身份和会话状态	81
7.2	前端环境变量	82
7.3	登录注册页面联调	83
7.4	发送消息的 UI 状态机	84
7.5	浏览器 F12 是前后端联调的现场证据	85
7.6	curl / Swagger / 前端三层测试法	86
7.7	前后端闭环现场演示脚本	87

7.1 前端改造集中在 API、身份和会话状态



链路 Day 2 页面保留，数据来源改为真实 API

- 增加统一 API Client，集中管理 Base URL、认证和错误处理。
- 增加认证状态：登录、退出、恢复当前用户、Token 过期处理。
- 把消息列表的数据源从假数组改为后端历史接口。
- 发送按钮改为调用真实对话接口并处理 loading/stream/error。

7.2 前端环境变量

统一 API Client 应处理什么

- 使用框架约定的公开变量保存后端 Base URL，例如本地 `http://localhost:8000/api/v1`。
- 公开变量会进入浏览器包，只能放可公开配置，绝不能放模型 API Key/数据库密码。
- 生产地址由构建/部署环境注入，不能把 localhost 散落在组件里。
- Base URL、JSON 序列化、Authorization Header。
- 非 2xx 响应转换为统一错误对象。
- 401 触发登录失效流程；支持请求取消与超时。
- 普通 JSON 请求与流式响应分别处理，不能使用同一种解析方式。

落点 普通 JSON 请求与流式响应分别处理，不能使用同一种解析方式。

7.3 登录注册页面联调

会话列表联调

- 表单前端校验提升体验，后端校验保证安全。
- 显示重复邮箱、密码错误、网络失败等不同状态。
- 登录成功后调用 /users/me 再次验证 Token 并加载当前用户。
- 页面加载时查询当前用户会话并按更新时间展示。
- 新建会话后使用后端返回的真实 ID 更新路由/状态。

7.4 发送消息的 UI 状态机

- `idle → sending/streaming → success` 或 `error/cancelled`。
- 发送期间避免重复提交；支持停止生成或重试失败消息。
- 临时消息与数据库真实 message ID 在完成后对齐。

现场验证·学员要拿出证据

- Network 核对请求参数
- 后端日志定位业务分支
- 数据库查询确认最终状态

落点 临时消息与数据库真实 message ID 在完成后对齐。

7.5 浏览器 F12 是前后端联调的现场证据

- Network：请求是否发出、URL/方法是否正确、Payload 是否匹配。
- Headers：Origin、Content-Type、Authorization 是否存在。
- Response/Preview：状态码、错误结构和流式内容。
- Console：只用于补充，不能把所有 4xx/5xx 都误判为 CORS。

联调检查顺序

```
Network -> Request URL -> Method / Status ->
Request Headers -> Request Payload -> Response Body
Console -> CORS / JS 异常后端日志 -> 路由 / SQL / 上游模型
```

验收 命令、版本、端口和实际执行文件位置都能被现场验证

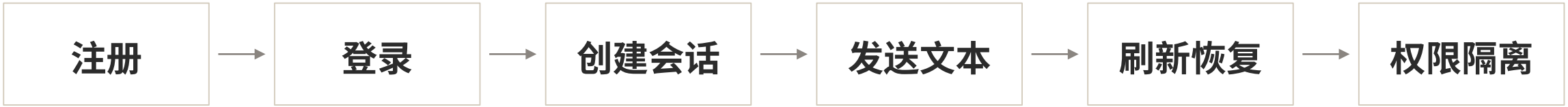
7.6 curl / Swagger / 前端三层测试法

典型错误：前端显示成功但后端失败

- curl 验证网络与后端最底层行为。
- Swagger 验证接口流程与 Schema。
- 前端验证用户体验、状态管理和跨域。
- 若 Swagger 成功而前端失败，优先检查前端请求构造与 CORS。
- 忘记检查 HTTP 状态码，拿到错误 JSON 仍追加“成功”气泡。
- 乐观 UI 没有失败回滚，数据库实际没有记录。
- 使用假 fallback 回复掩盖模型错误；演示看似顺畅但系统不可审计。

落点 使用假 fallback 回复掩盖模型错误；演示看似顺畅但系统不可审计。

7.7 前后端闭环现场演示脚本



链路 现场演示按稳定顺序推进，失败能定位

- 新用户注册、登录、创建会话、发送问题并看到流式回答。
- 打开数据库客户端展示用户、会话和两条消息记录。
- 刷新页面、重新登录、重启后端，再次打开历史会话。
- 第二个账号修改 URL/会话 ID，证明无法读取第一个账号的数据。

08

AI/Vibe Coding 常见工程问题与排错

AI 可以快速生成代码，工程证据决定它是否真的完成

8.1	AI 最常见的假后端：内存数组冒充数据库	89
8.2	AI 最常见的“假登录”	90
8.3	AI 会编造包、方法和过期教程	91
8.4	AI 容易吞掉异常	92
8.5	数据库常见故障	93
8.6	流式响应常见故障	94
8.7	高质量提示词先锁定边界、步骤与验收	95
8.8	让 AI 帮你 Debug 时必须提供什么	96

8.1 AI 最常见的假后端：内存数组冒充数据库

页面能动、接口返回 200，都不能证明数据真的持久化

假完成	表面现象	一步戳穿
内存数组	增删改看似正常	重启后端后再次查询
localStorage	刷新仍能看到	换浏览器或清空站点数据
Mock / setTimeout	聊天有回复	查看 Network 是否调用真实模型链路
假登录	页面显示用户名	用错误密码、过期 Token、跨用户资源测试
假数据库	GUI 里有表	查看本次请求对应 SQL 与真实新增记录

原则 任何“完成”都要能被 Network、日志、数据库和重启测试共同证明

8.2 AI 最常见的“假登录”

AI 最常见的密钥泄露

- 任意密码都能进入；前端把邮箱写进状态就视为登录。
- 后端签了 Token，但受保护接口从不校验。
- 有认证没有授权：登录后可读取所有用户会话。
- 把 Key 写进前端、源码、截图、日志、README 或 Git 历史。
- `.gitignore` 是最后一道基本防线，不代表已经提交的 Key 自动消失。
- 一旦泄露立即到供应商后台吊销/轮换，不只是删除本地文件。

落点 一旦泄露立即到供应商后台吊销/轮换，不只是删除本地文件。

8.3 AI 会编造包、方法和过期教程

AI 容易一次改太多文件

- 包名相似但不存在；API 已改名；不同大版本代码混在一起。
- 让 AI 先读取当前 `pyproject.toml/package.json/pom.xml` 与官方文档。
- 使用实际安装结果、类型检查和测试输出判断代码是否可用。
- 前端、后端、数据库、配置同时大改，出错后无法定位。
- 按垂直切片推进：health → DB 连接 → 注册 → 登录 → 会话 → 消息 → 模型 → 验收。
- 每一步通过小验收并提交 Git，再进入下一步。

落点 每一步通过小验收并提交 Git，再进入下一步。

8.4 AI 容易吞掉异常

同步/异步代码混用问题

- `except Exception: return {"success": true}` 会把真实故障伪装成成功。
- 用户响应应安全简洁，服务端日志要保留可定位的错误类别与堆栈。
- 不允许空 catch、无日志 fallback 或永远返回 200。
- async 并不会自动让阻塞数据库驱动/SDK 变成非阻塞。
- 阻塞调用会拖慢并发；课堂可先选一致、简单方案，生产再按测量优化。
- 不让 AI 为了“高性能”同时引入异步 ORM、线程池和队列而增加不可控复杂度。

落点 不让 AI 为了“高性能”同时引入异步 ORM、线程池和队列而增加不可控复杂度。

8.5 数据库常见故障

接口常见故障

- Server 未启动、端口不对、账号无权限、数据库不存在。
- URL 中密码特殊字符未编码、字符集不一致、迁移没执行。
- 连接池耗尽或 Session 未关闭；先读最早出现的根因异常。
- 404：路径或 /api/v1 前缀错误；405：HTTP 方法错误。
- 422：JSON 字段/类型与 Schema 不匹配；401：Token Header 缺失或过期。
- 500：去后端日志查根因，再让 AI 修改对应服务代码。

落点 500：去后端日志查根因，再让 AI 修改对应服务代码。

8.6 流式响应常见故障

重复消息与重复扣费

- 前端把流式响应当 `response.json()`；中文被错误分块解码。
- 代理/服务器缓冲导致“最后一次性出现”；连接中断未发送结束事件。
- 数据库只保存了部分回复；要明确中断状态与最终提交时机。
- 用户连点、网络重试和后端自动重试可能触发多次模型调用。
- 前端禁用重复提交只是体验措施，后端还需要请求 ID/幂等检查思维。
- 记录模型请求状态，不能把未知结果简单重放。

落点 记录模型请求状态，不能把未知结果简单重放。

8.7 高质量提示词先锁定边界、步骤与验收

- 先读现有目录、依赖、接口与 Day 2 代码，保留已完成的 UI。
- 先输出最小变更计划与验收命令，再按单一垂直切片实施。
- 明确技术栈、版本、接口 Schema、表结构、安全边界与禁止项。
- 完成后运行格式化、类型检查/测试，并报告实际输出和未完成项。

提示词四层结构

角色：你是资深后端工程师
任务：实现一个垂直切片
约束：技术栈、目录、接口契约、安全边界
步骤：先计划，再逐文件修改
验收：curl / 测试 / SQL 证据
输出：变更摘要与风险

验收 命令、版本、端口和实际执行文件位置都能被现场验证

8.8 让 AI 帮你 Debug 时必须提供什么

- 复现步骤、预期行为、实际行为、完整首个错误、相关日志。
- 操作系统、运行时版本、启动命令、相关依赖版本。
- Network 请求/响应和最小相关代码；隐去密码、Token 与 Key。
- 让 AI 先提出根因假设和验证方法，再按证据改配置。

现场验证·学员要拿出证据

- 给出可稳定复现的步骤
- 提供完整报错与环境信息
- 只修改最小范围并重新验证

落点 让 AI 先提出根因假设和验证方法，再按证据改配置。

09

课后作业与评分

作业验收围绕真实持久化、真实身份和真实模型闭环

9.1	课后作业：让智能客服真正可对话	98
9.2	基础必做验收 2：真实身份与隔离	99
9.3	提高项	100
9.4	评分只看能否拿出工程证据	101
9.5	一票否决/必须返工项	102
9.6	课程收尾：十个问题检验是否真正理解	103

9.1 课后作业：让智能客服真正可对话

基础必做验收 1：真实持久化

- 基于 Day 2 的前端，接入自己的后端和 MySQL。
- 必须实现注册、登录、新建会话、发送消息、读取历史。
- 必须使用真实模型 API；模型 Key 只能在后端环境变量中。
- 发送一轮问题后，数据库中可查到 user 与 assistant 两条消息。
- 刷新页面、关闭浏览器、重启后端后，重新登录仍能看到历史。
- 不允许仅使用内存、localStorage、JSON 文件或前端 mock 作为主存储。

落点 不允许仅使用内存、localStorage、JSON 文件或前端 mock 作为主存储。

9.2 基础必做验收 2：真实身份与隔离

基础必做验收 3：真实接口质量

- 密码只保存哈希；错误密码不能登录；重复邮箱有正确错误码。
- A、B 两个账号的数据隔离，修改会话 ID 不能越权读取。
- 模型 Key、数据库密码、JWT Secret 不出现在 Git 与前端产物。
- API 路径、方法、输入输出和状态码合理，Swagger 可用。
- 前端有加载、空、成功、失败状态，不把错误显示成成功。
- README 能让同学在新环境完成安装、迁移、启动和验证。

落点 README 能让同学在新环境完成安装、迁移、启动和验证。

9.3 提高项

提交材料

- 流式输出、停止生成、失败重试与幂等请求。
- 限流、超时、幂等请求、请求 ID 与更完整的故障恢复。
- 会话重命名/删除、分页、自动标题、Token/成本统计。
- 最小自动化测试和更完整的日志/请求 ID。
- Git 仓库：`frontend/`、`backend/`、迁移、`.env.example`、README。
- 2–4 分钟录屏：注册 → 登录 → 对话 → 查数据库 → 刷新/重启恢复 → 双用户隔离。
- 接口截图/导出、数据库表结构与关键记录截图。
- 一页项目说明：架构、技术栈、最难问题、排错证据、下一步计划。

落点 一页项目说明：架构、技术栈、最难问题、排错证据、下一步计划。

9.4 评分只看能否拿出工程证据

基础功能先达到闭环，再用提高项拉开差距

维度	可验证证据	分值
基础闭环	注册登录、会话 CRUD、真实模型、刷新恢复	35
数据库	表设计、约束、迁移、应用账号、真实记录	20
接口质量	OpenAPI、状态码、结构化错误、CORS、安全边界	15
工程质量	分层、配置、README、锁文件、Git 提交	15
流式与可靠性	事件协议、中断处理、超时重试、限流与幂等	10
展示表达	演示顺序、故障复盘、架构与取舍说明	5

返工 密钥泄露、明文密码、跨用户越权、假持久化任一出现，基础项必须重做

9.5 一票否决/必须返工项

- 密码明文存储；模型 Key 出现在前端或仓库。
- 用假回复冒充真实模型；用内存/localStorage 冒充数据库。
- 登录后可读取其他用户会话。
- 项目无法按 README 启动，且未说明依赖和环境。

现场验证·学员要拿出证据

- 功能可操作并覆盖失败分支
- 数据可查询且重启后仍存在
- 身份隔离与密钥管理可证明

落点 项目无法按 README 启动，且未说明依赖和环境。

9.6 课程收尾：十个问题检验是否真正理解

能完整回答，才说明已经掌握后端、数据与模型链路

1. 为什么刷新不丢数据不能只靠 React/Vue 状态?
2. 为什么模型 API Key 不能放前端?
3. 401 与 403 有什么不同?
4. JWT 为什么不能保存密码?
5. ORM 与数据库迁移分别解决什么?
6. 为什么应用不应使用 MySQL root?
7. 为什么有登录仍可能发生越权?
8. 一条聊天消息经过哪些组件?
9. 模型失败时数据库应该留下什么状态?
10. Day 4 的 RAG 将插入今天的哪一层?

表达 回答时说清边界、数据流、失败路径和验证证据

10

环境安装命令附录

提供可直接执行的安装命令、版本检查与连通性自检

10.1	Windows 终端与安装前检查	105
10.2	Windows：安装 uv 与 Python 3.12	106
10.3	macOS/Linux：安装 uv 与 Python 3.12	107
10.4	Windows：MySQL 8.4 LTS 安装步骤	108
10.5	macOS：MySQL 8.4 LTS 安装步骤	109
10.6	创建课程数据库与最小连通性验证	110

10.1 Windows 终端与安装前检查

- 统一使用 Windows 10/11 64 位；命令区分 PowerShell 与 CMD，复制时去掉 $/PS> 提示符。
- PowerShell 执行 `Get-Command uv, python, mysql -ErrorAction SilentlyContinue` 检查旧环境。
- 系统中已有多版本时，先记录 `where.exe uv`、`where.exe python`、`where.exe mysql`，再有依据地调整 PATH。
- 安装完成必须关闭并重开终端，必要时重启 VS Code，让新 PATH 生效。

Windows 安装前检查

```
Get-Command uv,python,mysql `
-ErrorAction SilentlyContinue
where.exe uv
where.exe python
where.exe mysql
```

10.2 Windows：安装 uv 与 Python 3.12

- 官方脚本：`powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"`；也可用 `winget install --id=astral-sh.uv -e`。
- 新终端执行 `uv --version`、`uv python install 3.12`、`uv python list`。
- 项目中执行 `uv python pin 3.12`、`uv run python --version`；不依赖系统 `python` 命令是否指向商店占位符。
- 企业电脑无法执行脚本时，使用官方发布的可执行文件/winget，并让管理员处理策略，不建议永久关闭安全策略。

Windows · uv / Python 3.12

```
winget install --id=astral-sh.uv -e
uv --version
uv python install 3.12
uv python pin 3.12
uv run python --version
```

10.3 macOS/Linux：安装 uv 与 Python 3.12

- 官方脚本：`curl -LsSf https://astral.sh/uv/install.sh | sh`；macOS 也可 `brew install uv`。
- 根据安装提示重开终端或加载 shell 配置，再执行 `uv --version`。
- 执行 `uv python install 3.12`、`uv python pin 3.12`、`uv run python --version`。
- Apple Silicon 与 Intel 的可执行路径可能不同，用 `which uv` 和 `uname -m` 辅助排错。

macOS / Linux · uv / Python 3.12

```
curl -LsSf https://astral.sh/uv/install.sh | sh
uv --version
uv python install 3.12
uv python pin 3.12
uv run python --version
```

10.4 Windows：MySQL 8.4 LTS 安装步骤

- 下载官方 MySQL 8.4 Windows x64 MSI，使用 MySQL Configurator 创建 Windows Service。
- 选择开发机配置、TCP 端口 3306、强 root 密码；记录服务名，设置随系统启动。
- PowerShell 可用 `Get-Service *mysql*` 检查状态；客户端执行 `mysql -u root -p`。
- 若 `mysql` 命令不存在但服务正常，把 MySQL `bin` 加入 PATH，或先使用完整路径/Workbench 连接。

Windows · MySQL 8.4 LTS

```
Get-Service *mysql*
mysql --version
mysql -u root -p
SELECT VERSION();
SHOW VARIABLES LIKE 'port';
```

10.5 macOS：MySQL 8.4 LTS 安装步骤

- 在“官方 DMG/PKG”与 Homebrew 中选定一种安装方式，并使用对应的服务管理命令。
- 官方包可在系统设置的 MySQL 面板管理服务；Homebrew 安装则用对应 `brew services` 管理。
- 执行 `mysql --version` 后仍要 `mysql -u root -p`，并运行 `SELECT VERSION();` 验证 Server。
- Apple Silicon 找不到客户端时检查 `/usr/local/mysql/bin` 或 Homebrew 前缀，并只添加实际存在的目录到 PATH。

macOS · MySQL 8.4 LTS

```
mysql --version
mysql -u root -p
SELECT VERSION();
# Homebrew 方案示例
brew services list
```

10.6 创建课程数据库与最小连通性验证

- root 登录后创建 customer_service，字符集选择 utf8mb4；创建仅用于本项目的本地应用账号。
- 只对 customer_service.* 授予课程所需权限，退出 root 后用应用账号重新登录。
- 依次执行 SELECT VERSION();、SHOW DATABASES;、USE customer_service;，证明版本、身份和权限正确。
- 最后让 FastAPI 执行一个数据库健康检查；“GUI 能连”与“应用能用 DATABASE_URL 连”是两个验收项。

数据库与账号验证

```
CREATE DATABASE customer_service CHARACTER SET utf8mb4;
CREATE USER 'customer_app'@'localhost' IDENTIFIED BY '***';
GRANT ALL ON customer_service.* TO 'customer_app'@'localhost';
SHOW GRANTS FOR 'customer_app'@'localhost';
```

全栈对话闭环完成

HELLO@DRAGONAI.CN · DRAGONAI.CN
2026